Day - 5

08 May 2026 09:48

1. CI/CD in openshift
2. Devops & git
3. cloud Native openshift
4. Optimization & cost awareness. ←
5. Use -case
6. Post Assessment.

1. CI/CD in openshift –
   1. openshift Pipeline (Tekton)
   2. Build → Test → Deploy flow
   3. git integration.

(1) What is CI/CD?

CI – Continuous Integration.

Developer Pushes code
↓
System automatically Build & test the code.

git Push → Build Image → Run test → Pass / fail

Continuous Deployment / Delivery:–

After Successful build & test
↓
Deploy the code in the env.

Ex:- Build Passed → Image → openshift.

2. CI/CD Workflow:–

Developer
↓
git Push
↓
Clone Source code
↓
Build Container Image
↓
Run Tests
↓
Push Image to Registry
↓
Deploy in openshift

Deploy in openshift

↓

Route expose the Apk

↓

Service

↓

Pod → container.

High level — git → Build → Test → Deploy.

Openshift - Tekton Building Blocks.

Main —

1. Task
2. TaskRun
3. Pipeline.
4. Workspace
5. Trigger
6. Event listner
7. Trigger Template
8. Trigger Building.

① Task - collection of steps.

② Task Run — Task : Test - app
step :
Run npm test

Task
- step1
- step2
- step3

} Task Run.

③ Pipeline:- collection of Task arranged in proper order.

Pipeline
- clone Image — T1
- Build Image — T2
- run-test — T3
- Deploy Image. — T4

④ Pipeline Run:- Actual execution of a pipeline.

Pipeline - Recipe

Pipeline Run - execution.

⑤ Workspace:- shared storage used by task.

Clone - source task download code

↓

Same code is used by Test task

↓

Same code is used by Test task

↓

Same code is used by build task.

Github / GitLab / BitBucket

↓

Webhook / Manual Trigger

↓

Openshift Pipeline Run.

↓

Task run1 : Clone

↓

Task run2 : Build

↓

Task run3 : Test

↓

Task run 4 : Deploy

↓

Openshift : Deployment / Route.

→ CI/CD Architecture in openshift

Build → Test → Deploy Flow

↓

Convert code into Image (S2I, Dockerfile, Buildconfig, Buildah Task)

↓

Run Unit Test, Integration Test, Security check or Quality check (Sonar Scan, Trivy, npm test, mvn test, py test)

→ update openshift app. with new Image / configuration

Developer Push

↓

Github Repo

↓

Pipeline Triggers (Webhook / Manual)

↓

Clone code.

↓

Build Image

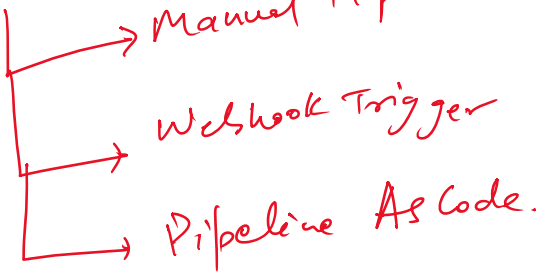
↓

Run Test

↓

Push Image → Deploy to Openshift → Verify Rollout.

Git Integration:-



(1.) Manual:-

pipeline run parameter: git: ——————→

(2.) Webhook Trigger:-

Git Push

↓

GitHub Webhook

↓

Openshift event Listner

↓

Pipeline start.

(3.) Pipeline As Code- stored inside Git repo.

.tekton/ pipeline.yaml.

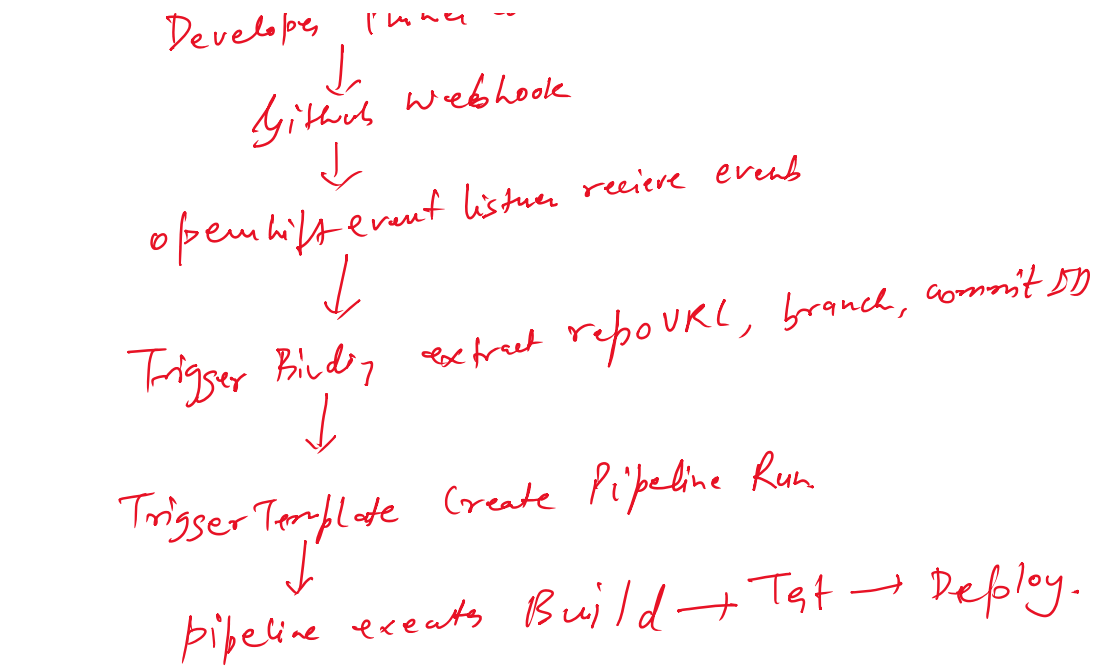
Developer Commit Code

↓

Developer Push Code to GitHub

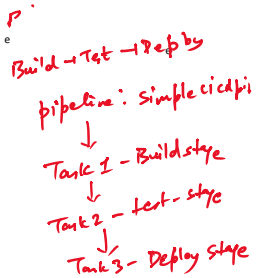
↓

Webhook



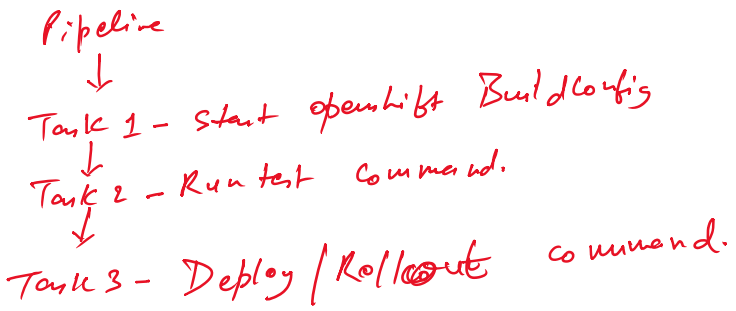

```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: simple-message-task
spec:
  params:
  - name: message
    type: string
  steps:
  - name: print-message
    image: registry.access.redhat.com/ubi9/ubi -minimal:latest
    script: |
      #!/bin/sh
      echo "Message received from pipeline:"
      echo "$(params.message)"
EOF
```

```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: simple-cicd-pipeline
spec:
  params:
  - name: message
    type: string
  tasks:
  - name: build-stage
    taskRef:
      name: simple-message-task
    params:
    - name: message
      value: "Build stage completed"

  - name: test-stage
    runAfter:
    - build-stage
    taskRef:
      name: simple-message-task
    params:
    - name: message
      value: "Test stage completed"

  - name: deploy-stage
    runAfter:
    - test-stage
    taskRef:
      name: simple-message-task
    params:
    - name: message
      value: "Deploy stage completed"
EOF
```




```
cat <<'EOF' | oc create -f -
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: simple-cicd-pipeline-run-
spec:
  pipelineRef:
    name: simple-cicd-pipeline
  params:
  - name: message
    value: "Starting CI/CD pipeline"
EOF
```

① Real CI/CD pipeline

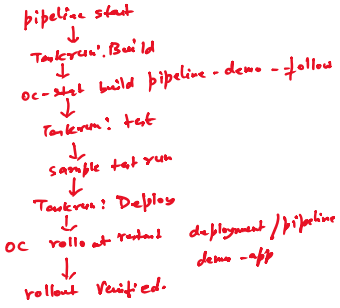



```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: openshift-build-task
spec:
  params:
  - name: buildconfig
    type: string
  steps:
  - name: start-build
    image: quay.io/openshift/origin-cli:latest
    script: |
      #!/bin/sh
      echo "Starting OpenShift build..."
      oc start-build $(params.buildconfig) --follow
EOF
```

```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: test-task
spec:
  steps:
  - name: run-tests
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    script: |
      #!/bin/sh
      echo "Running sample tests..."
      echo "Unit test 1 passed"
      echo "Unit test 2 passed"
      echo "All tests passed"
EOF
```

```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: openshift-deploy-task
spec:
  params:
  - name: deployment
    type: string
  steps:
  - name: rollout
    image: quay.io/openshift/origin-cli:latest
    script: |
      #!/bin/sh
      echo "Restarting deployment..."
      oc rollout restart deployment/$(params.deployment)
      oc rollout status deployment/$(params.deployment)
EOF
```

```
cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: build-test-deploy-pipeline
spec:
  params:
  - name: buildconfig
    type: string
  - name: deployment
    type: string

  tasks:
  - name: build
    taskRef:
      name: openshift-build-task
    params:
    - name: buildconfig
      value: $(params.buildconfig)

  - name: test
    runAfter:
    - build
    taskRef:
      name: test-task

  - name: deploy
    runAfter:
    - test
    taskRef:
      name: openshift-deploy-task
    params:
    - name: deployment
      value: $(params.deployment)
EOF
```



rollout verify etc.

(2) Devops & Gitops:-

(1) Jenkins integration concept
(2) Gitops. introduction
(3) Image promotion across the environment.

Traditional CI/CD

Developer → git → Jenkin/openshift Pipeline → Build → Test → deploy.

Gitops:-

Develops → git → ArgoCD watches git → openshift status update automatically.

Image promotion:-

Build once → Tag image as dev → Promote to test → Promote to Prod.

Jenkins:- CI/CD Automation Tool.

Jenkins + openshift flow

Developer Pushes Code
↓
github/gitlab Webhook trigger jenkins
↓
Jenkin pull the source code
↓
Jenkin run build/test

↓

Jenkins Build the container Image

↓

Image pushed in registry

↓

Jenkin run oc command

↓

openshift deployment updated.

Git Repo

↓ webhook

Jenkins

↓ oc login | SA

openshift API servs

↓

Project | Namespace

↓

Build Config | Deployment | Route | Service.

* gitops:- Developer chays yaml in git

↓

Pull req. review

↓

Merge to main Brand

↓

ArgoCD detech chays

↓

ArgoCD applies these chays in openshift

↓

Cluster state match Git.

Git

- deployment.yaml
- service.yaml
- route.yaml
- configmap.yaml

↓ watches

ArgoCD / openshift gitops

↓ sync

openshift cluster.

↓

Application Resources.

Git & cluster same

↓

status: synced

↓

change to cluster

↳ status: out of sync

ArgoCD sync your

status: synced.

(3.) cloud Native openshift

- openshift on AWS/Azure
- cluster upgrade & patching, Basics

SAAS

cluster upgrade & patching, Basic

Operator driven lifecycle.

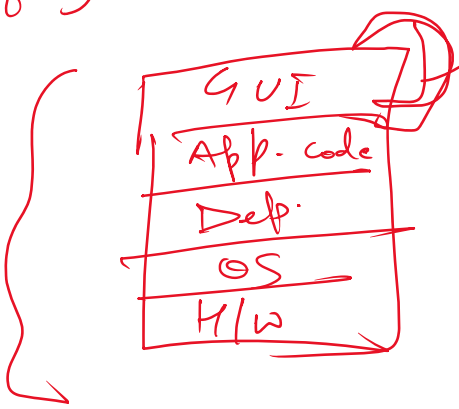
SAAS

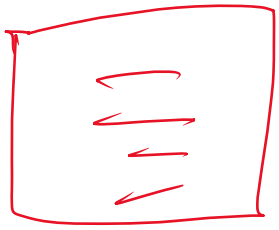
App.

(1) Managed

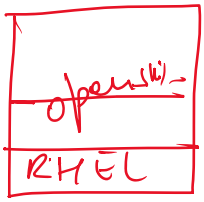
(2) SAAS.

Managed App





On-prem



AWS

Main upgrade compt:—

(1) cluster

(2) Machine configs

(3) cluster operator

(4) Node updates

(5) Control plane.

(6) Worker node

(7) operator.

operator lifecycle Manager:—

Operator available in catalogs

Operator available in catalogs

↓

Admin Install

↓

Subscription created

↓

Install Plan generated → Operator Installed

Operator Installed → Custom Resource → Handle upgrades.

* optimization & Cost Awareness:-

1. Resource optimization
2. Performance tuning.
3. Cost control strategies.

optimization:- CPU, Memory, Disk, Pods, resources

High level

Application

↓

Pod resources

↓

Node resources → cluster capacity → cloud cost

Resources:-